S.Y.B.Sc.(Data Science)
Sem- IV

Subject: Data Structure-II

Unit1 : Tree

By,

Mrs. Reshma Masurekar

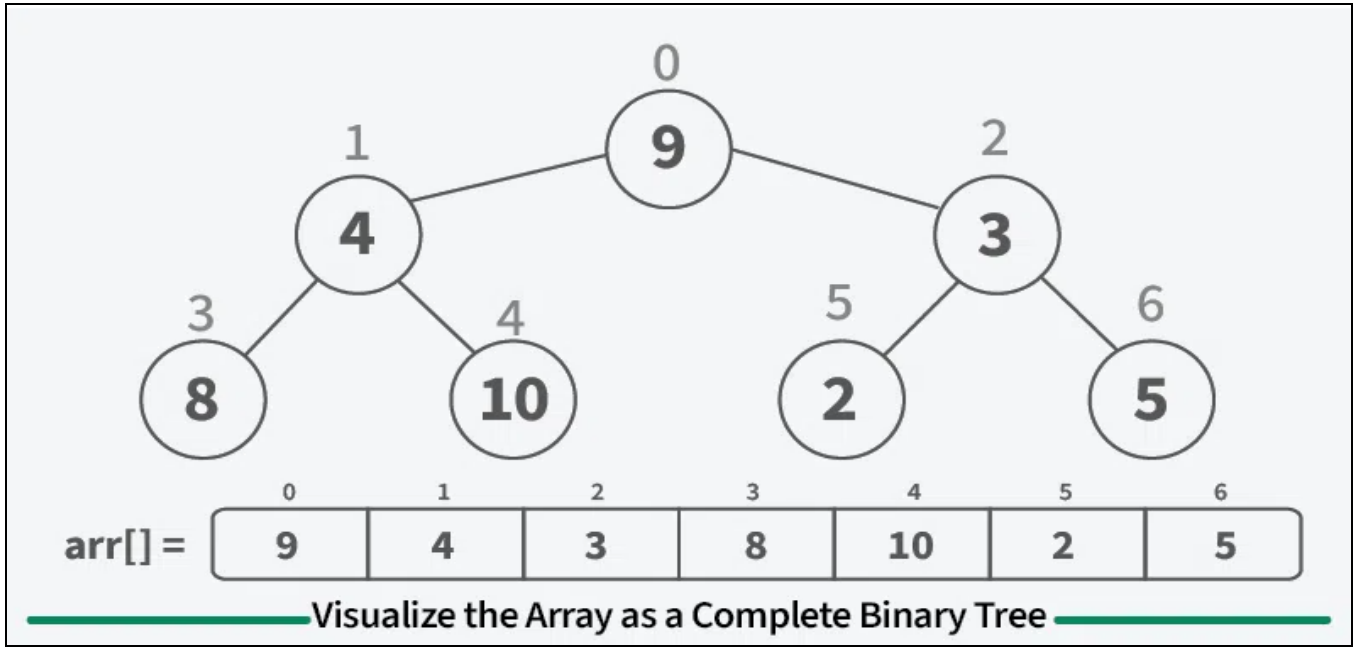
Dr. D. Y. Patil Arts, Commerce and Science College,
Pimpri, Pune 18

Heap Sort

- It is a comparison-based sorting algorithm that uses a Binary Heap data structure. It works similarly to Selection Sort, where we repeatedly extract the maximum element and move it to the end of the array.
- **Heap Sort Algorithm**
- Heap Sort works in two main phases:
  1. Build a Max Heap from the input data.
  2. Extract elements from the heap one by one, placing the maximum element at the end each time.
- **Algorithm Steps:**
  1. Rearrange array elements to form a Max Heap.
  2. Repeat until the heap has only one element:
  3. Swap the root (maximum element) with the last element of the heap.
  4. Reduce heap size by 1.
  5. Heapify the root to restore heap property.
  6. The array is now sorted in ascending order.

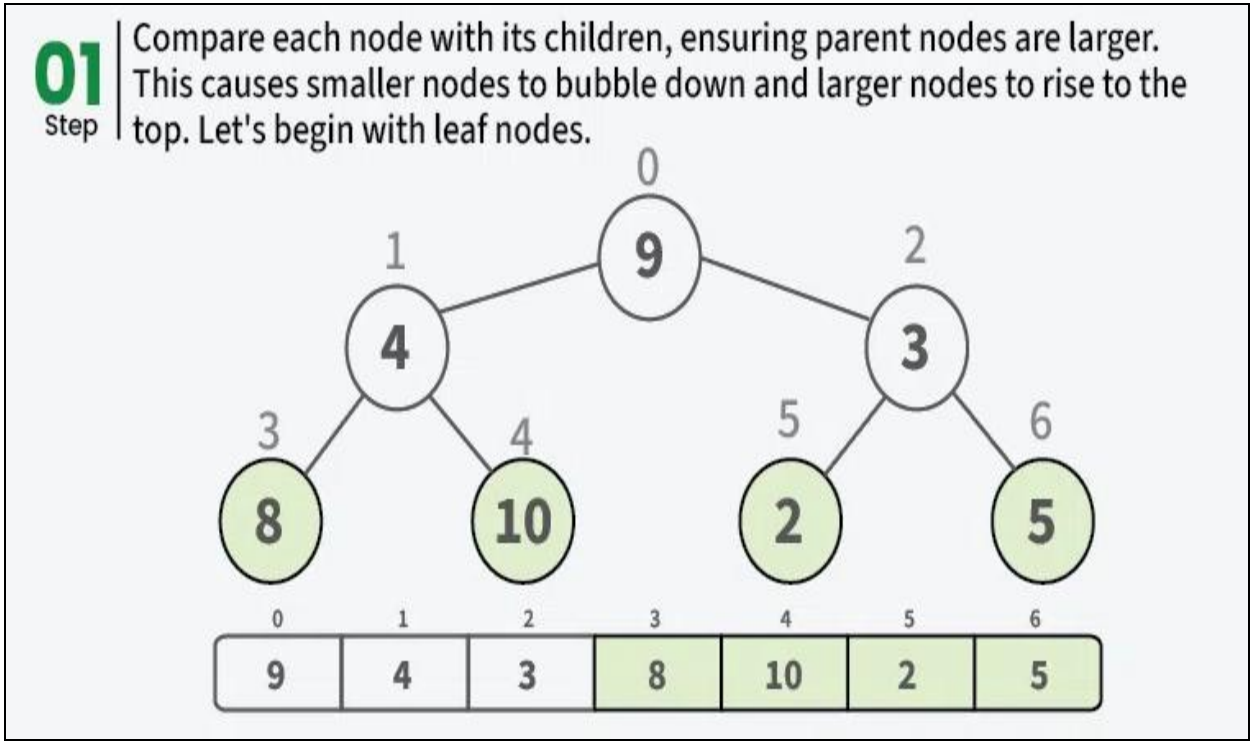
How Heap Sort Works

- **Step 1: Represent the Array as a Binary Tree**
- For an array of size n,
  - Root is at index 0
  - Left child of node **i: 2*i + 1**
  - Right child of node **i: 2*i + 2**



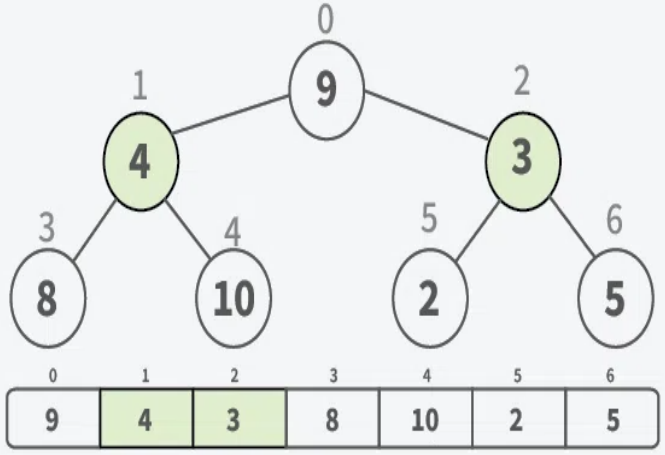
Step 2: Build a Max Heap

- **Step 2: Build a Max Heap**
- Convert the array into a max heap where each parent is larger than its children.



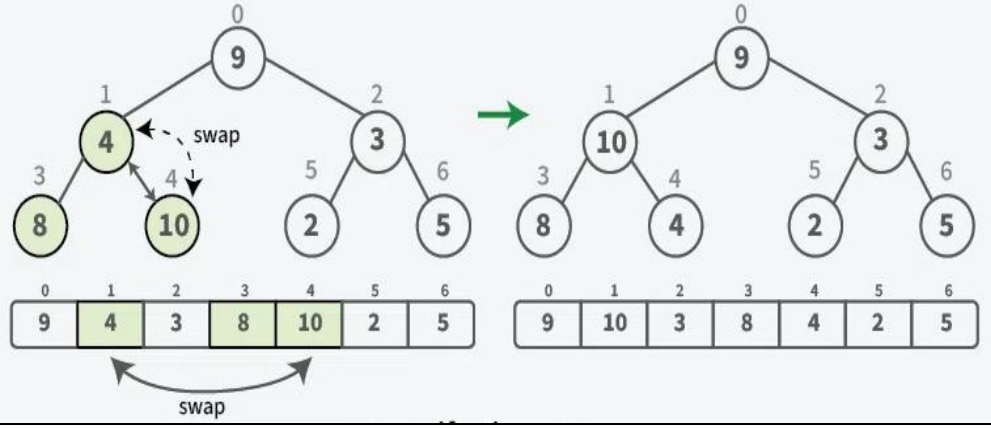
02 Step

Let's look in the next upper level (node 4 and 3)



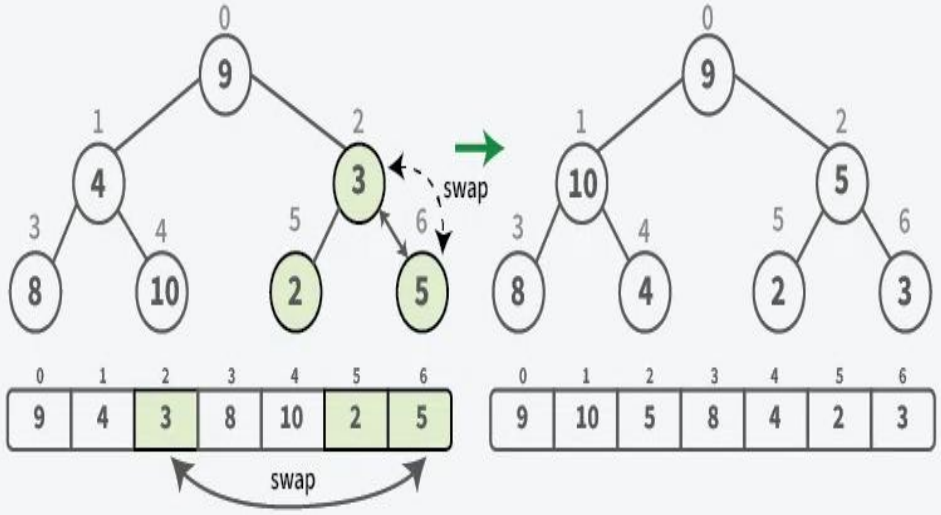
03 Step

Node 4 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.



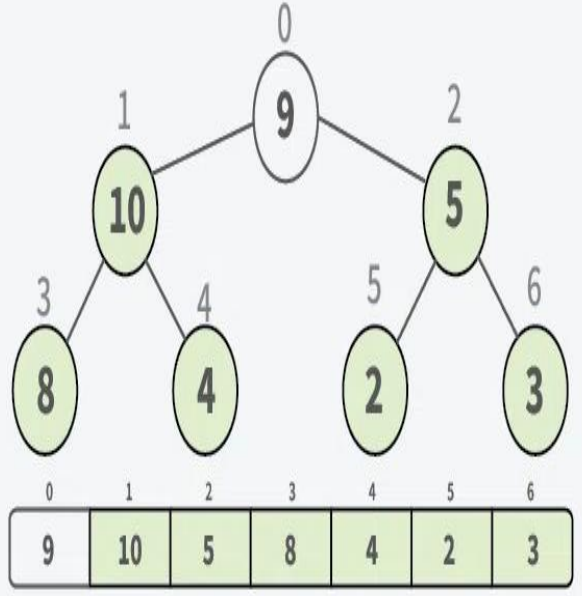
04 Step

Check for the next node (3) in current level. Since, it has a larger child, so swap it to ensure the parent has a larger value.

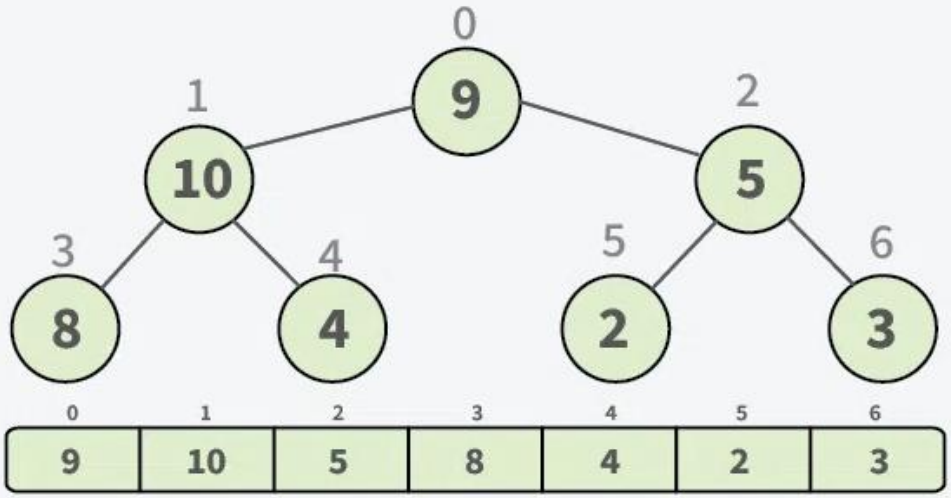


05 Step

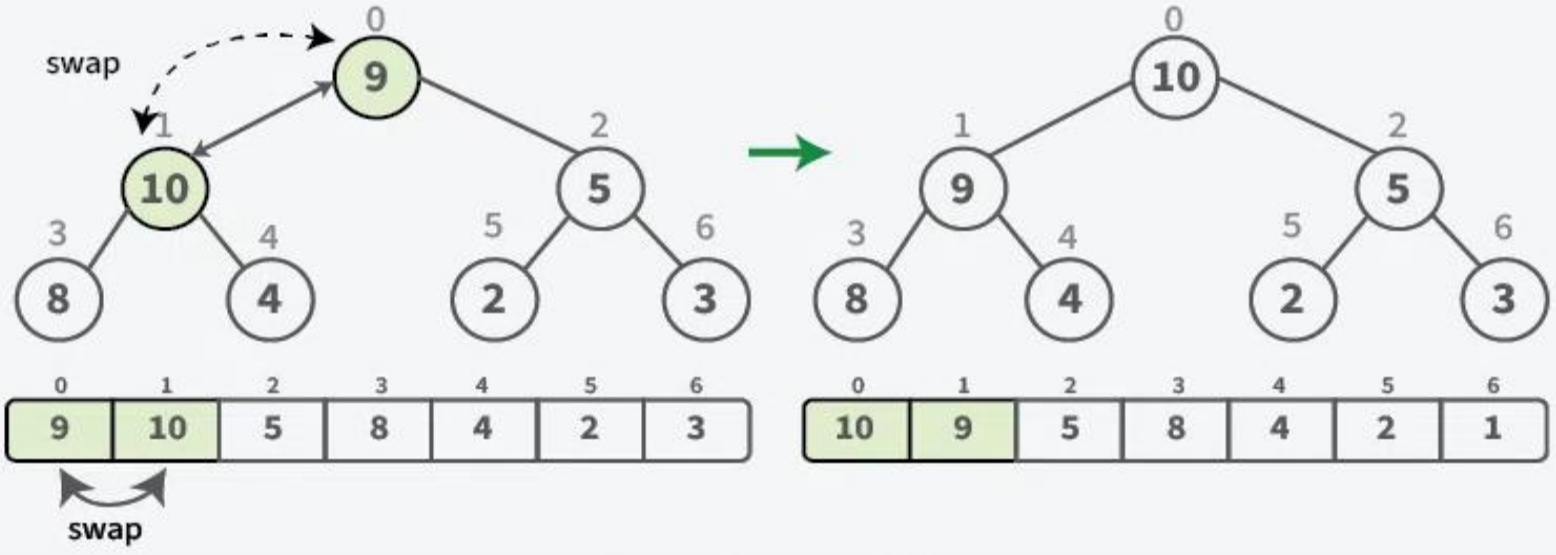
After the completion of current level, we have now two smaller valid max heap



06 Step | Let's move to the next upper level. Here we've node 9 at the root.

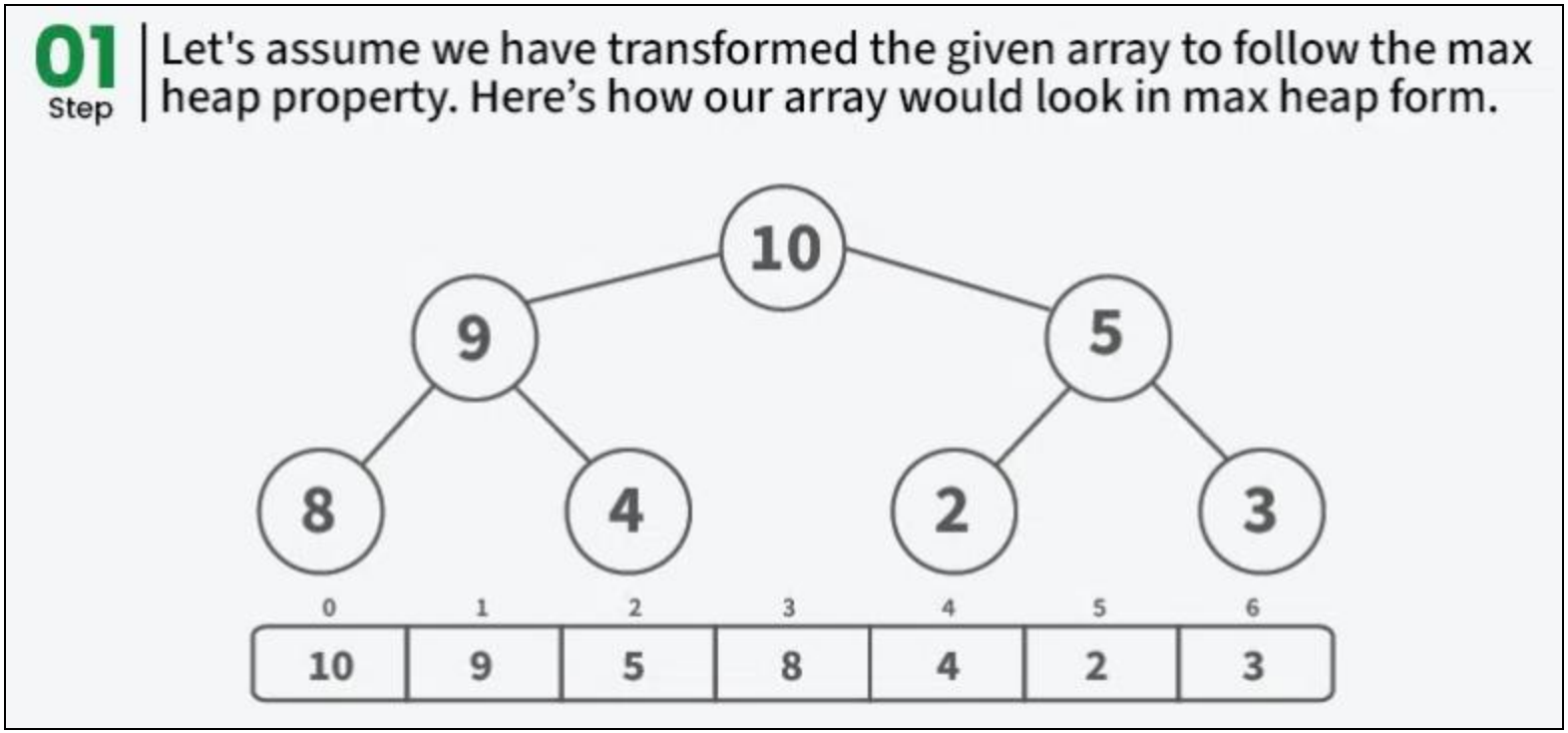


07 Step | Node 9 is smaller than its child node (10), so swap it with the larger child to maintain the property that the parent should be larger than its children.



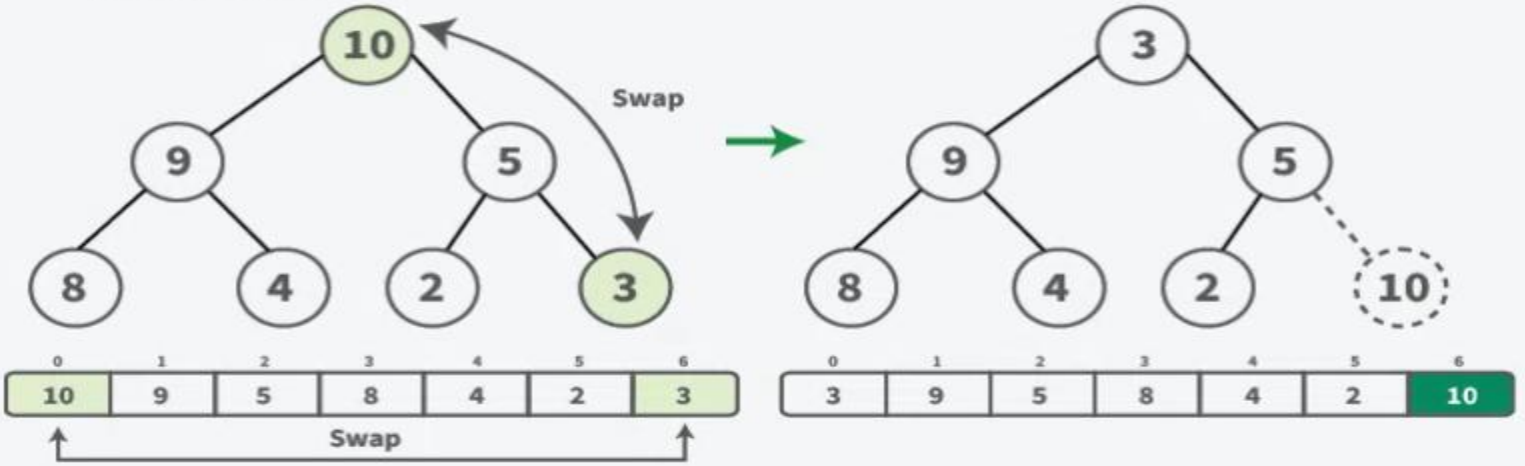
Step 3: Sort the array by placing largest element at end of unsorted array.

- Below are the detailed steps to sort the array:
  - Swap the root (maximum) with the last element.
  - Reduce heap size and heapify the root again.
  - Repeat until the heap has one element left.



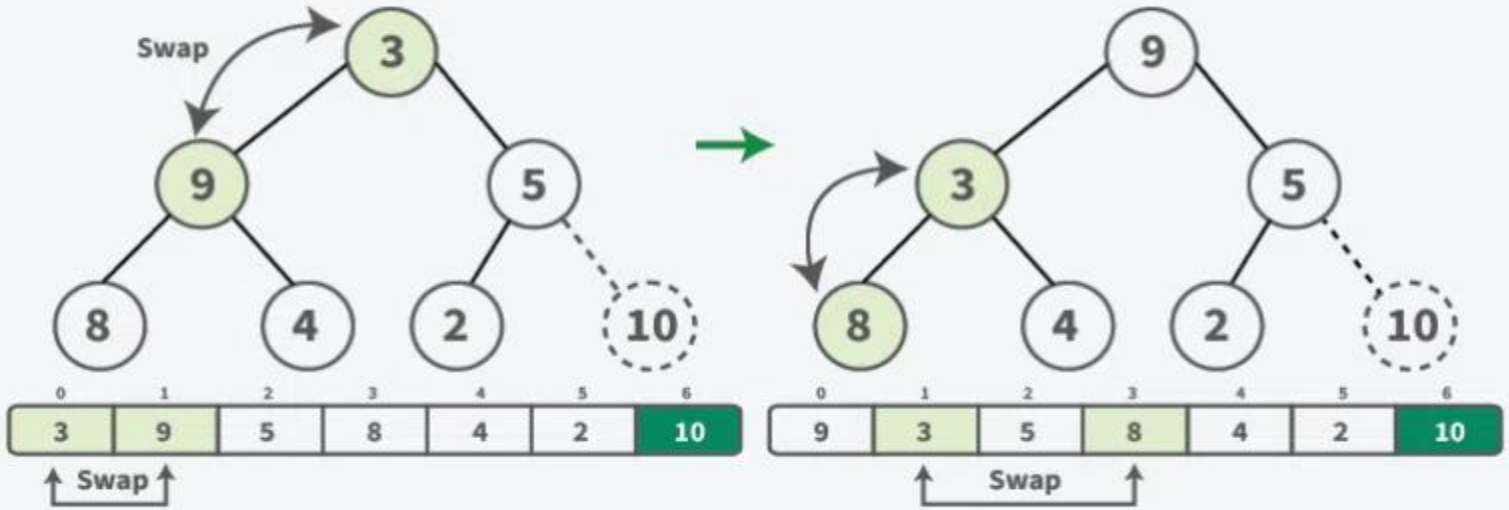
02 Step

Swap the maximum element (10) with the last element (3) in the unsorted array. Decrease the size of the heap by one (ignore the last element, as it is now sorted).



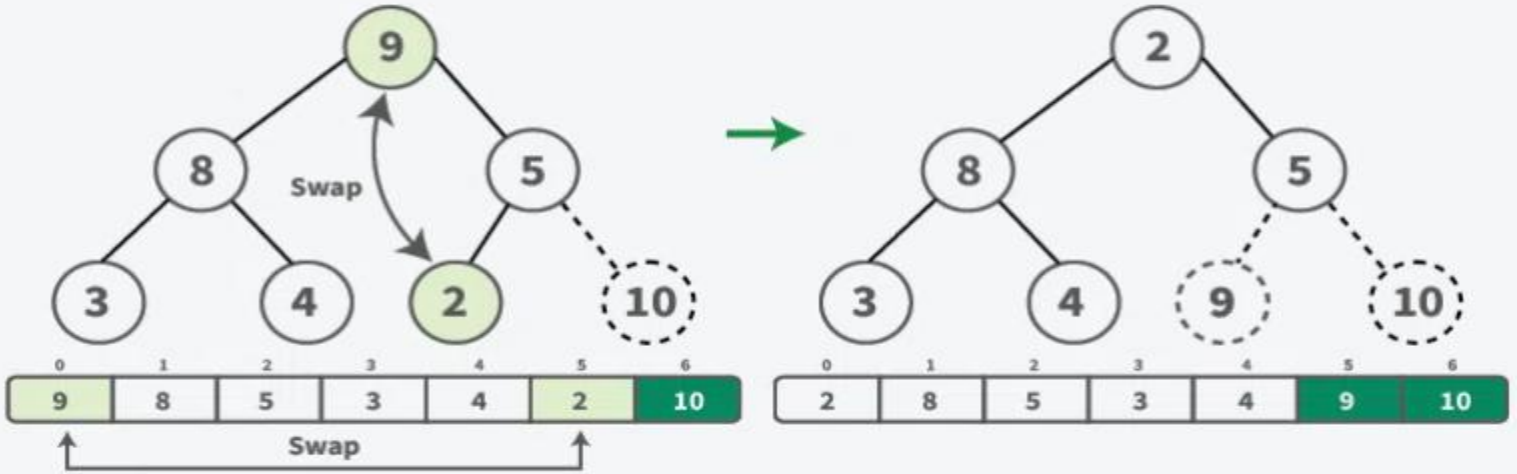
03 Step

Now, root violate the max-heap property, So, heapify it.
Swap node 3 with its largest child (9).
Still node 3 have larger children, so swap it with largest one (node 8).



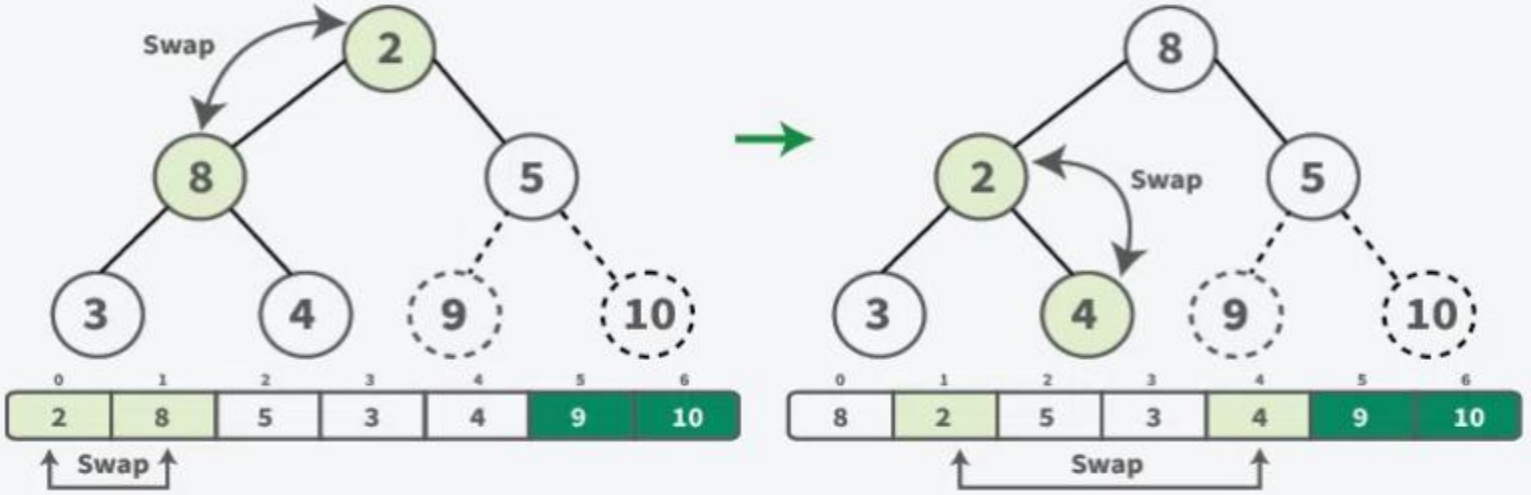
04 Step

Now, we have a max heap. Swap the maximum element (9) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the second last element as it's now sorted).



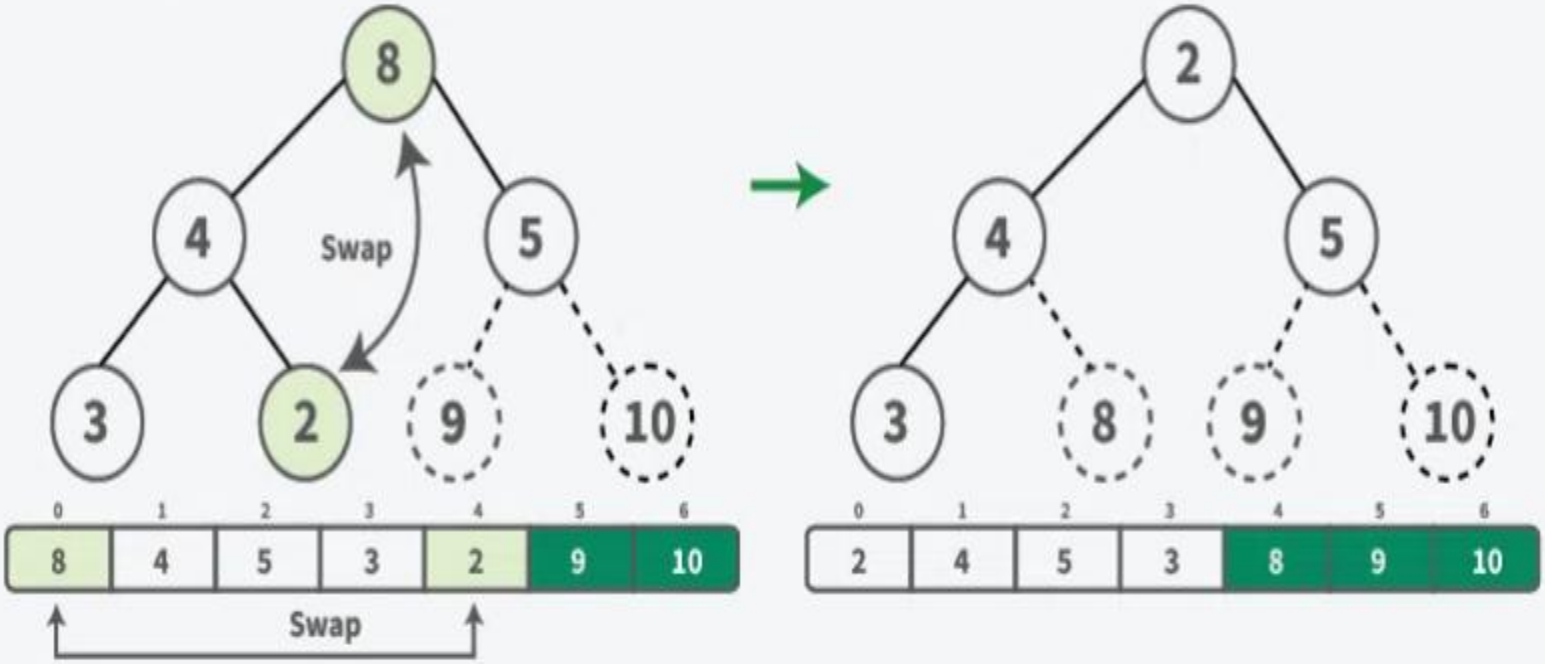
05 Step

Now, root violate the max-heap property, So, heapify it.
Swap node 2 with its largest child (8).
Still node 2 have larger children, so swap it with largest one (node 4)



06 Step

Now, we have a max heap. Swap the maximum element (8) with the last element (2) in the unsorted array, then decrease the heap size by one (ignoring the third last element as it's now sorted).



```
def heapify(a, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and a[left] > a[largest]:
        largest = left
    if right < n and a[right] > a[largest]:
        largest = right
    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)

def heapSort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
    for i in range(n - 1, 0, -1):
        a[i], a[0] = a[0], a[i]  # Swap max to end
        heapify(a, i, 0)
```